

Лабораторная работа №6 Применение OpenSSL

Теоретические основы

В лабораторной работе рассмотрим систему OpenSSL, вызываемую через командную строку.

OpenSSL – криптографическая библиотека с открытым исходным кодом. Она основана на SSLeay, написанной Эриком Янгом (Eric A. Young) и Тимом Хадсоном (Tim Hudson), которые неофициально закончили работать над ней в декабре 1998 года, когда они начали работать в проекте RSA Security. Название SSL переводится, как система безопасных сокетов (secure socket layer). Библиотека содержит реализации популярных алгоритмов шифрования, хеширования и протоколов обеспечения защищенного обмена данными (SSL/TSL, основанные на сертификатах X.509).

OpenSSL используется практически всеми сетевыми серверами для защиты передаваемой информацией. Существует программное API SSL (SSLEAY), позволяющее создавать безопасные сокет с шифрованием передаваемых данных в собственных разработках.

OpenSSL поддерживает большое количество различных алгоритмов и форматов представления данных, поэтому использование данной команды достаточно сложно. Внутри OpenSSL существуют отдельные компоненты, отвечающие за то или иное действие, для получения списка доступных компонентов можно вызвать openssl с параметрами list-standart-commands. Можно также получить список доступных алгоритмов хеширования (list-message-digest-commands) и алгоритмов шифрования (list-cipher-commands).

OpenSSL может использоваться во множестве случаев и умеет, в частности, выполнять следующие задачи:

Создавать и управлять ключами ассиметричных криптографических алгоритмов RSA и DSA – команды **rsa**, **dsa**, **dsaparam**

Создавать сертификаты формата x509, запросы на получение, восстановление сертификата – команды **x509**, **req**, **verify**, **ca**, **crl**, **pks12**, **pks7**.

Зашифровывать данные с помощью симметрического или асимметрического шифрования – команды **enc**, **rsautl**

Вычислять хэш-значения различных типов – команда **dgst**

Работать с S/MIME – команда **s/mime**

Проверять работу серверов и клиентов ssl – команды **s_client**, **s_server**.

Существуют также ряд вспомогательных утилит ssl:

openssl speed [список_алгоритмов_хеширования_или_шифрования]:

тестирование скорости различных алгоритмов, если запускать без параметров, то тестируются все алгоритмы; алгоритмы внутри списка разделяются пробелом, например

openssl speed md5 rsa idea blowfish des 3des sha1.

В конце работы выводится общая скорость работы различных алгоритмов (в 1000-х байт в секунду), для обработки различной длины блоков.

openssl rand [-out file] [-rand file] num:

генерация num случайных байт.

openssl ciphers [-ssl2] [-ssl3] [-tls1] NAME:

вывод доступных алгоритмов для обеспечения уровня безопасности NAME, где NAME - это символическое название группы алгоритмов.

Обычно используются значения

LOW - алгоритмы низкого уровня безопасности(<128 бит)

MEDIUM - алгоритмы среднего уровня стойкости(128 бит)

HIGH - алгоритмы высокой стойкости(> 128 бит)

ALL - все алгоритмы

NULL - алгоритмы без шифрования.

Обычно в настоящее время используются алгоритмы групп MEDIUM и HIGH, которые ещё долго не смогут быть взломаны прямым перебором. Можно также вывести список алгоритмов из нескольких групп, разделив их: (например, MEDIUM:HIGH).

Задание

1. Установить OpenSSL на виртуальную машину и ознакомиться с возможностями библиотеки (команда «?»).
2. Выполнить тестирование скорости выполнения различных алгоритмов шифрования.
3. Создать криптографические ключи. Выбрать несколько произвольных файлов и выполнить.
 - а) шифрование (зашифрование и расшифрование) посредством различных симметричных алгоритмов;
 - б) шифрование (зашифрование и расшифрование) посредством различных асимметричных алгоритмов;
 - в) хэширование различных файлов различными алгоритмами (обязательно md5 и sha1).
4. Создать самоподписанный сертификат X509. Изучить состав сертификата и назначение его компонентов.
5. Оформить отчет. В отчет поместить:
 1. результаты тестирования производительности.
 2. времена шифрования;
 3. полученные хэш значения
 4. Сертификат и анализ его содержания.